上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/23

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

资源管理：调度与技术路线

模块 6 · 资源管理（2/3）

饮 水 思 源 • 爱 国 荣 校

SJTU

资源管理 – 调度与技术路线

本讲位置：从「给多少」到「放哪儿」

上一讲 · L-09

隔离与分配：配额表回答了「每个服务给多少」

遗留问题：给多少定了，放在哪台机器上由谁决定？放错了会怎样？

本讲问题

实例放在哪台机器上，才能既放得下、又不怕坏一台？

引例（设定）

开放日前一周，按 L-09 的配额表放好 4 个 Tomcat 和批处理，用流量回放演练。想加第 5 个 Tomcat，集群还剩 5.1 核、9.5 GB，新实例却一直 Pending（**症状 A：放不下**）；重启节点 2，2 个 Tomcat 同时下线，剩下 2 个每个要扛 4.35 核（**症状 B：一起坏**）

本讲结构

1 放得下：装箱

2 坏得起：故障域

3 补得回：优先级与重调度

4 技术路线

5 课程实践：放置方案

课程实践以一张「放置方案」回答引例

本模块 资源管理（L-09 ~ L-11）：隔离与分配（L-09）· 调度（本讲）· 伸缩（L-11）

来源：L-09 第 24、29 页；L-08 第 20 页；引例中的数字均为设定值

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、放得下：装箱

第十讲 · 调度与技术路线

饮 水 思 源 • 爱 国 荣 校

SJTU

资源管理 – 调度与技术路线

调度在做什么



规则 每个节点、每个维度都要满足 Σrequest ≤ 可分配容量（L-09 第 17 页）

调度器按 request 记账，不看实际用量：用量低于 request 的部分，别人也不能放进来

回顾 L-09 第 24 页：「如何放置是 L-10 的内容」

为什么需要调度器

2 ~ 3 个节点、6 ~ 7 个实例：可以在黑板上手算

Q-2 的三年后平台：约 70 台机器、34 个在线实例，另有离线作业，组合数爆炸

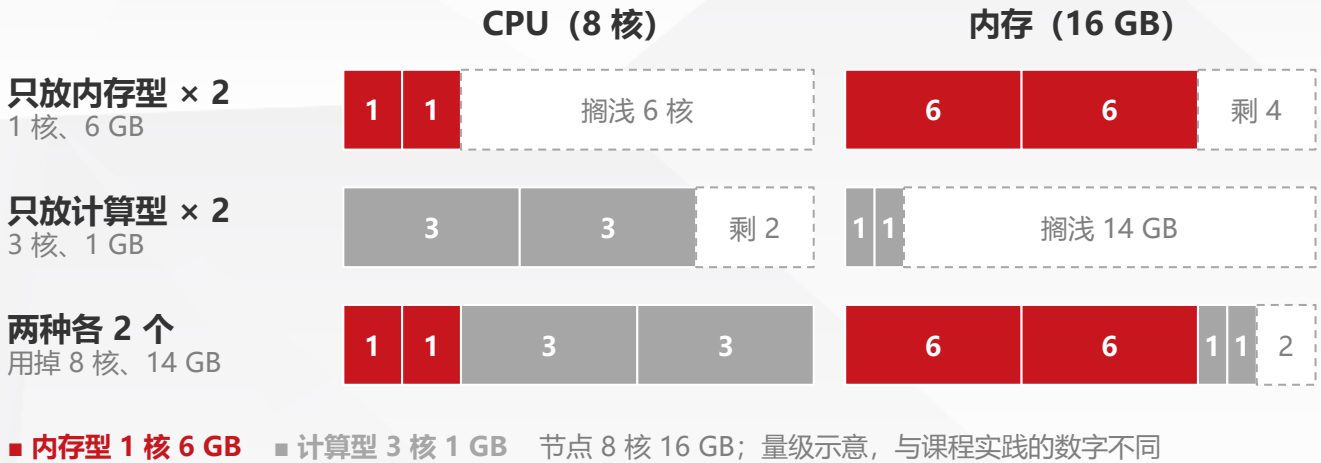
这是多维装箱问题（NP 难），没有已知的快速最优解法；调度器用启发式规则，在很短时间内给出「足够好」的放法

要点 调度器按 request 记账；request 定错，放法也跟着错

来源：Kubernetes docs (v1.37), Kubernetes Scheduler；L-09 第 17、24 页

放得下 · 装箱

多维装箱与碎片



两个概念

搁浅（stranded）：某一维先用完，其余维度的剩余放不下任何实例

碎片：集群总剩余够，但分散在多个节点上，没有一个节点单独放得下。例：3 个节点各剩 2 GB，放不下一个 3 GB 的实例

问题	原因	对策与代价
节点总剩余够，新实例仍 Pending	两维都要满足，剩余分散在多个节点	节点规格匹配实例形状（CPU : 内存 = 1 : 2、1 : 4 等）；形状互补的负载混放
节点 CPU 闲、内存满	实例形状与节点形状不一致	调整规格；或把实例拆小，代价是每个实例都有固定开销（L-06 第 7 页）
运行越久，碎片越多	实例增删后剩余分散	重调度整理（第 16 页），代价是被移动的实例要重启

要点 装箱是多维的：最先用完的那一维决定能放几个，其余维度的剩余就是浪费

来源：示意图为量级示意；L-06 第 7 页

过滤与打分



过滤去掉放不下或不允许放的节点；打分给剩下的排序，取最高分

全部被过滤掉 实例 Pending，事件写明原因（症状 A）：

0/2 nodes are available: 1 Insufficient cpu, 2 Insufficient memory. preemption: …

「1」「2」是节点数：1 个节点 CPU 不够，2 个节点内存都不够

资源打分策略	偏向	效果
LeastAllocated（默认，CPU 与内存等权）	已分配少的节点	**摊开**
MostAllocated	已分配多的节点	**装满**
RequestedToCapacityRatio	自定义曲线与权重	按需

默认的拓扑分布

未配置时，按节点 maxSkew 3、按可用区 maxSkew 5 尽量分散（ScheduleAnyway）

是「尽量」，不是保证（第 11 页）

要点 过滤决定「能不能放」，打分决定「放哪个更好」

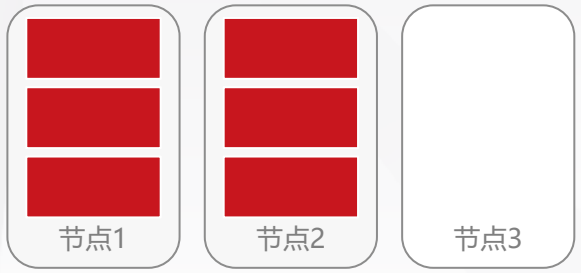
来源：Kubernetes docs (v1.37)：Scheduling Framework、Resource Bin Packing、Pod Topology Spread Constraints

放得下 • 两种策略

装满与摊开

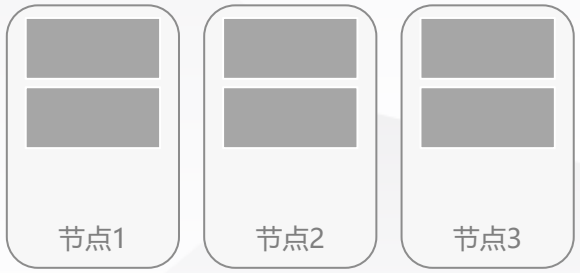
策略	好处	代价	适合
装满（bin packing）	节点少，可关掉空节点，省钱	坏一个节点丢得多；邻居干扰；突发时余量小	批处理、平时的低峰、可中断的负载
摊开（spreading）	每个节点留余量吸收突发；坏一个节点丢得少	节点多，每个都只用一部分	在线、延迟敏感的负载；开放日

装满：6 个实例放进 2 个节点



坏一个节点丢 3 个；空节点可关掉

摊开：6 个实例分到 3 个节点



坏一个节点丢 2 个；每个节点都有余量

同一个集群，两种策略并存

在线服务摊开；批处理填进剩余空间（形状互补，第 6 页）

回顾 L-09 第 8 页：利用率与延迟的冲突，在放置上再出现一次

要点 装满省的是机器，摊开换来的是余量和更小的故障半径

来源：示意图为量级示意；L-09 第 8 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、坏得起：故障域

第十讲 · 调度与技术路线

饮 水 思 源 • 爱 国 荣 校

SJTU

故障域：会一起坏的范围



并联公式的前提

1 – ∏(1 – A)（L-08 第 20 页）假设各副本独立失效

放在同一故障域的副本会一起坏：要按故障域算，不能按副本数算（症状 B）

云上看不见的一层

两台云主机可能落在同一台物理宿主机上

华为云的云服务器组：反亲和性策略把组内云主机放到不同物理机；故障域策略放到不同机架

要点 副本数只是数量；副本分布在几个故障域里，才决定一次故障带走多少

来源：L-08 第 20 页；华为云文档：弹性云服务器 ECS「管理云服务器组」

反亲和与拓扑分布

规则	含义	效果	代价
必须反亲和（podAntiAffinity，required）	同一拓扑域内最多一个	一域一个，最强	域的数量不少于副本数，否则 Pending
偏好反亲和（preferred）	尽量分开	不保证	资源紧张时可能挤在一起
拓扑分布约束（maxSkew 1，DoNotSchedule）	各域实例数相差不超过 1	均匀分布，一域可放多个	某个域满了，新实例 Pending
污点、容忍与节点亲和	只让某类实例使用某些节点	专用节点（第 13 页）	专用节点上的剩余空间别人用不了

拓扑域是什么

按节点：kubernetes.io/hostname；按可用区：topology.kubernetes.io/zone

Kubernetes 只认节点上的标签，看不见云主机下面的物理宿主机

例：3 个节点放 6 个副本

必须反亲和（按节点）：只放得下 3 个，其余 3 个 Pending

拓扑分布 maxSkew 1：放成 2、2、2

要点 反亲和管「别放在一起」，拓扑分布管「放得均匀」

来源：Kubernetes docs (v1.37)：Assigning Pods to Nodes、Pod Topology Spread Constraints、Taints and Tolerations

按「容量够」计算可用性

两种「可用」

服务还在：至少 1 个副本存活

容量够：至少 k 个副本存活，k 由资源账决定（L-07）；开放日要的是后者

n 个节点相互独立，每个可用性 p：

P(至少 k 个可用) = Σ C(n, i) × p^i × (1 − p)^(n − i)

对 i = k, k + 1, …, n 求和

例 • 回顾 Q-1 第四幕（进阶）

4 台各 99%，至少 3 台：$0.99^4 + 4 \times 0.99^3 \times 0.01 \approx 99.94\%$；至少 1 台：$1 - 0.01^4$，8 个 9

副本不是一台一个时：先列节点的存活组合，再数副本

例：4 个副本放在节点 A（3 个）、B（1 个），要求至少 3 个；p = 99%

节点 A	节点 B	概率	存活副本	容量够
活	活	p^2	4	是
活	坏	$p(1-p)$	3	是
坏	活	$(1-p)p$	1	否
坏	坏	$(1-p)^2$	0	否

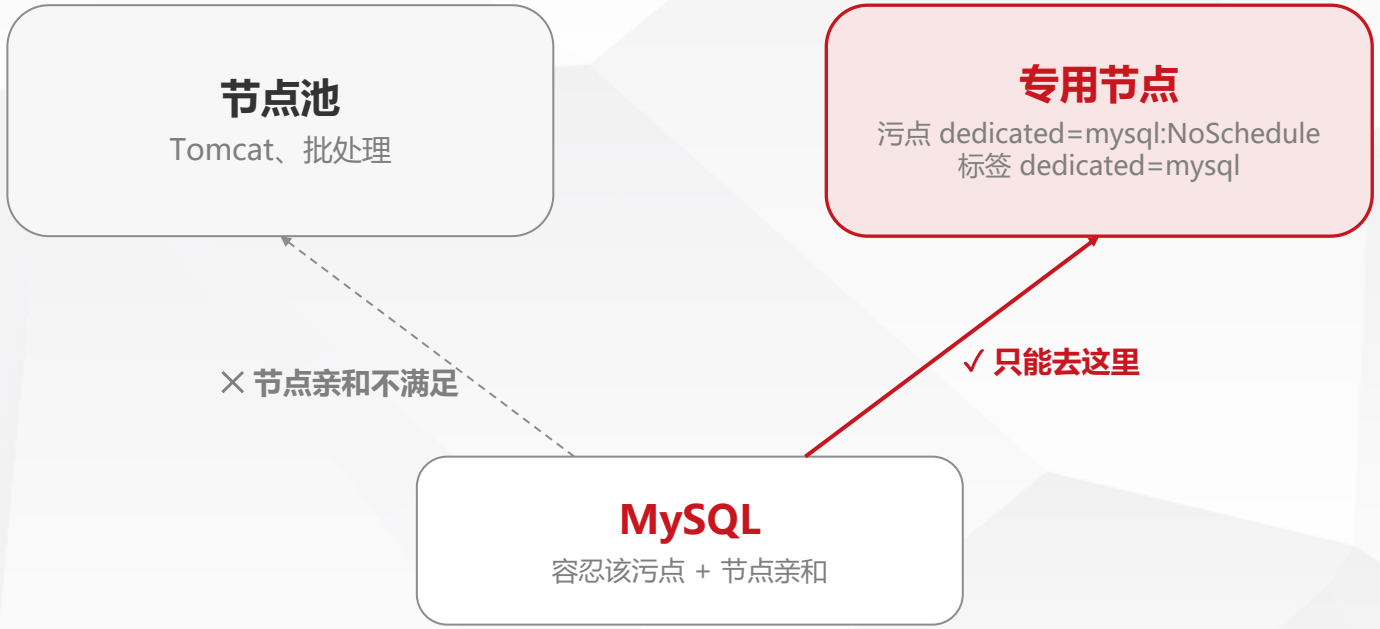
合计 $p^2 + p(1-p) = p = 99\%$

只要节点 A 活着就够：4 个副本的可用性，等于放了 3 个副本的那个节点的可用性

要点 可用性要按「容量够」来算；冗余要体现在故障域之间，而不只是副本数量

来源：L-08 第 20 页；思考题 1 第四幕；表为算术推演

有状态负载与专用节点



污点挡住别人（Tomcat 不容忍，进不来）；节点亲和把自己拉过去。两者都要：只有容忍，MySQL 仍可能被放进节点池

数据在哪儿，实例就得在哪儿

云硬盘只能挂到同一可用区的云主机上；换节点就是搬数据

不与在线无状态负载混部：fsync 敏感，隔离盲区多（L-09 第 15 页）

主从放在不同故障域

回顾 L-05 第 25 页

有状态组件的实例生命周期须较长，替换须谨慎

要点 无状态实例的放置随时可以改，有状态实例的放置是长期决定

来源：Kubernetes docs (v1.37), Taints and Tolerations；华为云文档：云硬盘 EVS「挂载云硬盘」

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、补得回：优先级与重调度

第十讲 · 调度与技术路线

饮 水 思 源 • 爱 国 荣 校

SJTU

优先级与抢占

1. 高优先级实例放不下（实例带 PriorityClass）
2. 选一个节点，挑出要驱逐的低优先级实例（nominatedNodeName）
3. 被抢占者按宽限期优雅停止（默认 30 s，L-05 第 24 页）
4. 腾出后放入；其间别处若有空位，也可能放到别处

代价	说明
被抢占者的工作丢失	需要检查点（L-09 第 28 页）
抢占不是瞬间完成	要等被抢占者优雅停止
只按 request 腾位置	被抢占者的 request 小，腾出的也少，不一定够
PodDisruptionBudget	只是尽力遵守，找不到别的办法时照样抢占

Borg 的做法（Verma 等，2015）

优先级分档：监控 > 生产 > 批处理 > 尽力而为

生产档之间不互相抢占：避免高的挤掉略低的、略低的再挤掉更低的，形成连锁抢占

资源回收：按实际用量估算每个任务的 reservation，把多出的部分给批处理等非生产任务；生产任务仍按 limit 放置

与 L-09 的区别

QoS 回答「资源紧张时谁先让」，优先级回答「放不下时谁让位」

选课平台：Tomcat 高优先级，批处理低优先级

要点 优先级是放不下时的让位规则，代价由被抢占的一方承担

来源：Kubernetes docs (v1.37), Pod Priority and Preemption；A. Verma et al., Borg, EuroSys 2015, §2.5、§5.5

补得回 • 重调度

放置会过时：节点故障之后

约 3 s：Nginx 摘除，流量切走
Q-1 第四幕

补建 → 调度 → 拉镜像 20 s → 启动加预热约 1 min



两个前提

剩下的节点要有空位，否则新实例 Pending

补回之前，剩下的实例要扛得住

为何默认这么慢

等待调短，恢复更快；但网络抖动时会误判、误驱逐

回顾 Q-1 第四幕：健康检查调得过敏，一次抖动全站 502 三分钟

其他让放置过时的情况

负载变化使节点冷热不均：重调度，代价是一次重启

节点维护先排空（drain）；PodDisruptionBudget 限制同时下线的副本数，只管主动下线，管不了节点故障（模块 7）

要点 流量几秒就能切走，容量要几分钟才能补回，而且要有空位；开放日的冗余必须提前放好

来源：Kubernetes docs (v1.37)：Taints and Tolerations、kube-controller-manager、Disruptions；L-05 第 30 页；拉镜像 20 s 为设定

阅读 • 公平

公平：多维资源怎么分

多维资源怎么分才公平

多个团队共享一个集群，任务形状不同；只按一种资源分，会偏向某一方

DRF（主导资源公平）

每个用户占比最大的那种资源是它的主导资源；分配时让各用户的主导份额尽量相等

选课平台只有一个团队，用不到；Q-2 的三年后平台（14 个服务、离线计算）会遇到

例（Ghodsi 等，2011）：集群 9 核、18 GB

A 的任务 1 核、4 GB；B 的任务 3 核、1 GB



A 得 3 个任务：3 核、12 GB；主导资源是内存，12 ÷ 18 = 2/3

B 得 2 个任务：6 核、2 GB；主导资源是 CPU，6 ÷ 9 = 2/3

两者的主导份额相等，都是 2/3

要点 多维资源的公平，看每个人最紧张的那一维

来源：A. Ghodsi et al., Dominant Resource Fairness, NSDI 2011

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、技术路线

第十讲 · 调度与技术路线

饮 水 思 源 • 爱 国 荣 校

SJTU

技术路线 • 演进

演进线：三问由谁回答

路线	给多少（粒度）	放哪儿（由谁放）	何时变（量级）	隔离	成本模型
物理机加脚本	整机	人工规划	采购，周到月	物理隔离	买机器，按峰值采购
虚拟化	虚拟机规格	虚拟化平台加人工	分钟（建虚拟机）	hypervisor	按机器（私有云）
云主机加弹性伸缩组	云主机规格	云平台按策略放置（可设反亲和组）	分钟（创建并启动）	hypervisor	按时长计费
容器编排	request、limit（L-09）	调度器（本讲）	秒到分钟（节点不够时还要加节点）	cgroup 加 namespace	按节点池计费，利用率靠装箱
Serverless 与托管服务	内存档位、请求	平台	毫秒到秒（冷启动，L-06 第 26 页）	平台沙箱	按调用次数与时长

回顾 L-06 第 26 页 那张表比较的是运行时（隔离、启动时间、密度）；这张表比较的是资源管理三问（L-09 第 5 页）由谁回答

「何时变」一列只给量级，L-11 再算

要点 每一步阀门更细、反应更快，三问的答案逐步交给平台；控制力和可见性也随之减少

来源：L-06 第 26 页；L-09 第 5 页；时间为量级示意

选课平台怎么选

负载	路线	理由	代价
Tomcat（在线无状态）	容器编排	秒级起停、按 request 装箱、可设拓扑分布	要有人维护集群，或用托管集群（如云容器引擎 CCE）
批处理（离线、可重跑）	同一集群、低优先级	填剩余空间、可被抢占	完成时间不确定
MySQL（有状态）	专用云主机，或托管数据库（如云数据库 RDS）	不进弹性层；fsync 敏感	少了弹性；托管则少了控制
平时只有几十 QPS 的接口	可考虑 Serverless（如函数工作流 FunctionGraph）	可缩到零，按调用计费	冷启动 100 ms ~ 数秒，开放日的突发不适合

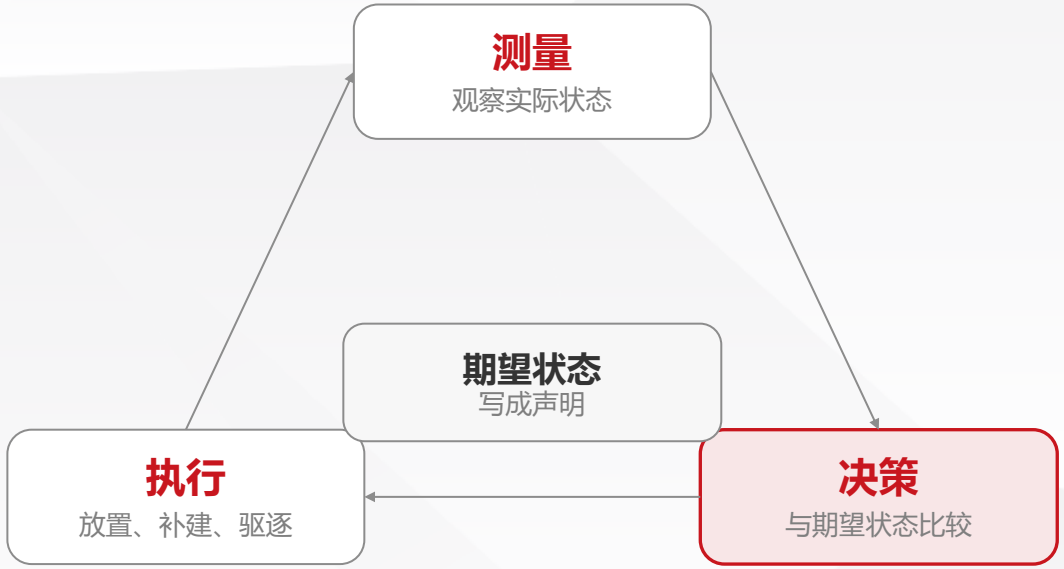
华为云的对应产品（L-01 的云平台） 弹性云服务器 ECS（含云服务器组）、弹性伸缩 AS、云容器引擎 CCE（托管 Kubernetes）、云容器实例 CCI（不用管节点的容器）、函数工作流 FunctionGraph

只列名称与定位，不讲配置和价格；用其他云平台的组按各自文档对照

要点 按负载类型选路线，一个平台里可以几条路线并存

来源：华为云文档：ECS、AS、CCE、CCI、FunctionGraph、RDS 产品介绍；L-06 第 26 页

控制环交给平台



本讲见过的两个控制环

调度器：观察待放的实例与节点 → 选节点 → 绑定

节点故障后的补建（第 16 页）：发现副本少了 → 补建 → 再调度

期望状态

把「6 个 Tomcat，各节点最多相差 1 个」写成声明，平台不断把实际状态拉回去；这是模块 7 声明式管理的起点

下一讲 L-11：期望状态里的副本数本身要随流量变化，这就是伸缩

要点 调度不是一次性的决定，而是一个持续运行的控制环

来源：Kubernetes docs (v1.37), Controllers；B. Burns et al., Borg, Omega, and Kubernetes, ACM Queue, 2016

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：放置方案

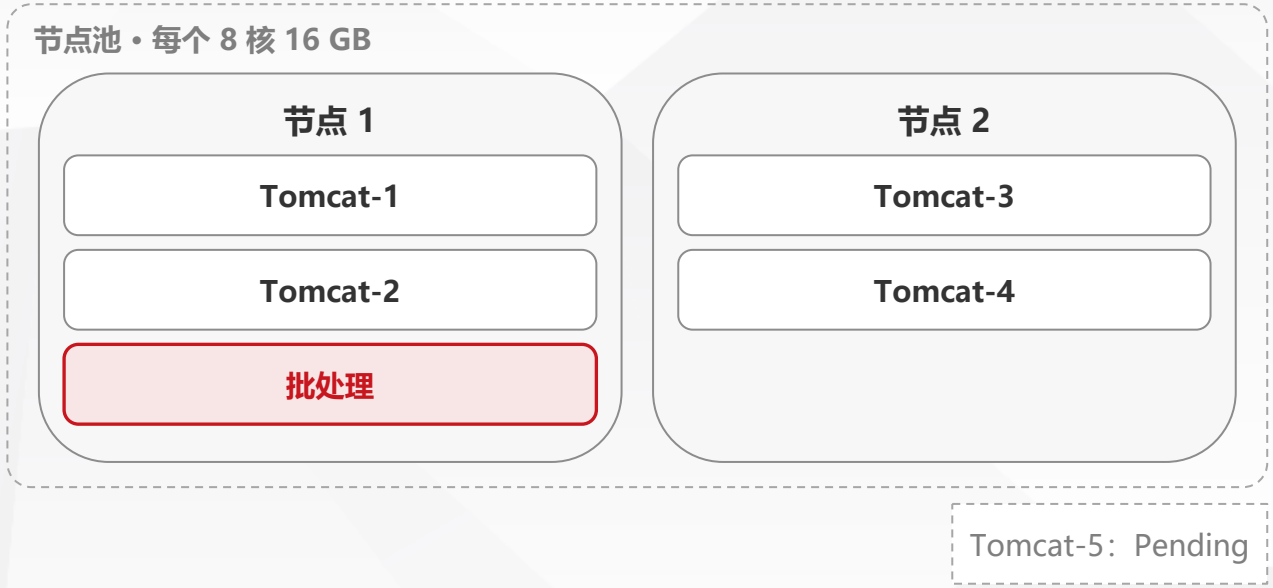
第十讲 · 调度与技术路线

饮 水 思 源 • 爱 国 荣 校

SJTU

课程实践 • 设定

参数设定与演练记录



症状 A 0/2 nodes are available: 1 Insufficient cpu, 2 Insufficient memory

症状 B 重启节点 2，只剩 2 个 Tomcat：稳态每个要扛 8.7 ÷ 2 = 4.35 核，超过 4 核 limit

沿用 L-09、L-07

Tomcat：request 2.6 核、5.5 GB，limit 4 核、5.5 GB；批处理：request 0.5 核、0.5 GB，limit 4 核、3 GB；MySQL 独占 4 核 8 GB，不在池内

Tomcat 共需：首 10 秒 10.2 核（2400 × 3 ms + 600 × 5 ms），稳态 8.7 核；每个按容量线承担 4 × 0.7 = 2.8 核，至少 4 个

新增（本讲设定）

节点可用性 99%（沿用 Q-1 第四幕），各节点独立失效

可以再加 1 个同规格节点（8 核 16 GB）

镜像拉取约 20 s；Nginx 健康检查约 3 s 摘除故障实例（Q-1 第四幕）

注 数值为设定，与 L-07 的资源账、L-09 的配额表自洽；节点故障后的判定与驱逐时间见第 16 页

来源：L-09 第 24、29 页；L-07 第 24、25 页；思考题 1 第四幕

课堂讨论

1 放不下

集群还剩 5.1 核、9.5 GB，为什么第 5 个 Tomcat 放不下？

除了加节点，还有哪些办法？各有什么代价？

2 坏得起

4 个副本放在 2 个节点上，「容量够」的可用性是多少？

要扛住一个节点故障，需要几个节点、几个 Tomcat，怎么放？用什么规则保证？

3 补得回

按第 2 题的方案，9:00 一个节点宕机。流量多久切走？它上面的 Tomcat 多久能在别处补回？

补回之前靠什么？

4 MySQL 进池

按第 2 题的方案，三个节点的剩余合计比 MySQL 的 4 核 8 GB 还多。

能不能把 MySQL 放进节点池，省下一台？

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

讨论要点（1） （2）

1 放不下：内存先满，剩余是碎片

每个节点按内存只放得下 2 个 Tomcat（3 × 5.5 = 16.5 > 16），按 CPU 放得下 3 个（3 × 2.6 = 7.8 ≤ 8）：每个节点搁浅 2.8 核（35%）

节点 1 剩 2.3 核、4.5 GB，节点 2 剩 2.8 核、5.0 GB：9.5 GB 分在两处，哪一处都不够 5.5 GB

办法与代价：加第 3 个节点，多一台机器；换 8 核 24 GB 的规格可放 3 个，内存仍剩 7.5 GB；内存 request 压到 5.3 GB，每节点放 3 个，余量由约 8% 降到约 4%，回到 L-09 第 14 页的 OOM 风险

挪走批处理：节点 1 变为 2.8 核、5.0 GB，仍差 0.5 GB，无效。追问：每节点装 3 个，坏一个就丢 3 个（第 2 题）

2 坏得起：副本数由放法决定

并联公式 $1 - 0.01^4$ 假设 4 个副本独立失效；同一节点上的 2 个会一起坏，要按节点算

「容量够」要求至少 4 个 Tomcat 存活。2 个节点各放 2 个：两个节点都得活，$0.99^2 = 98.01\%$

3 个节点各放 2 个（共 6 个）：至少 2 个节点存活即可，$0.99^3 + 3 \times 0.99^2 \times 0.01 \approx 99.97\%$

规则：按节点的拓扑分布，maxSkew 1，DoNotSchedule；用必须反亲和则一节点一个，需要 6 个节点

对照：3 个节点放 5 个（2、2、1），或 2 个节点各装 3 个，都仍是 98.01%

结论 副本数 = 容量所需的 4 个 + 单个节点上最多的 2 个 = 6 个

讨论要点（3） （4）

3 补得回：靠预先放好的冗余

流量：Nginx 约 3 s 摘除；剩 4 个 Tomcat，首 10 秒每个 10.2 ÷ 4 = 2.55 核，低于 2.8 核的容量线，扛得住。这正是放 6 个的意义

补回：50 s 判定不健康，再过 300 s 驱逐，约 6 分钟后才开始补建；剩下的节点最多剩 2.8 核、5.0 GB，放不下 5.5 GB，新实例 Pending，直到节点恢复或加节点（L-11）

抢占批处理只能腾出 0.5 核、0.5 GB，仍不够：调度按 request 腾位置

能否把 300 s 调短：恢复更快，但网络抖动时会误驱逐（Q-1 第四幕）

4 MySQL 进池：放不下，也不宜放

放不下：没有一个节点单独放得下 4 核 8 GB（每个最多剩 5 GB），碎片不能合并

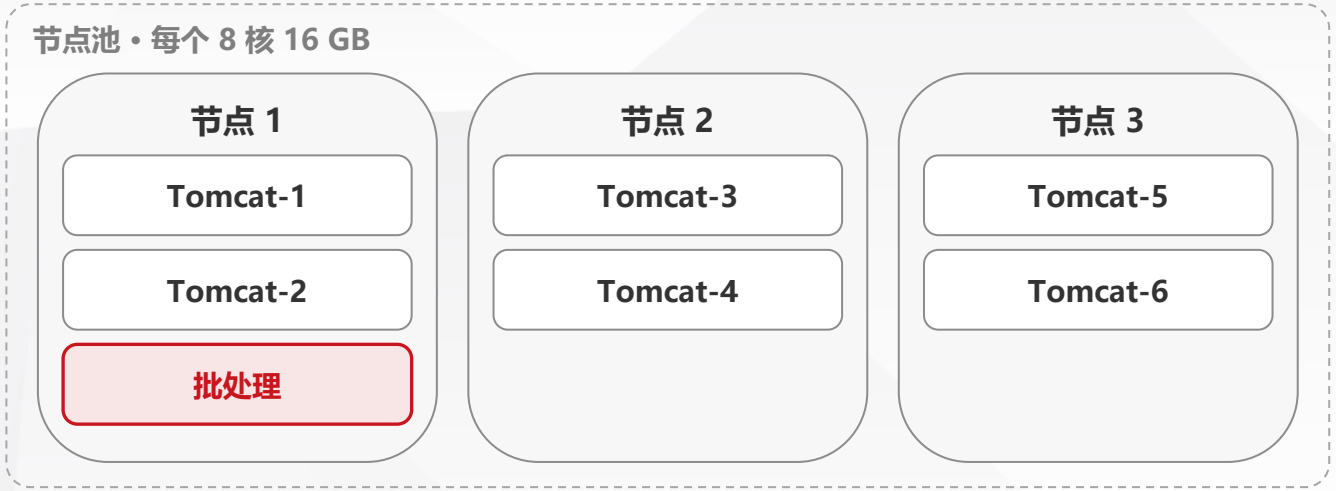
即使换更大的节点也不宜：有状态，换节点就要搬数据；对 fsync 敏感，与 Tomcat、批处理混部落在隔离盲区（L-09 第 15 页）

故障域耦合：一个节点坏了，会同时带走 2 个 Tomcat 和数据库，第 2 题的 99.97% 不再成立

做法：独占，用专用节点（污点、容忍加节点亲和）；主从放在不同故障域

结论 开放日前 10 分钟靠预先放好的冗余，不靠重调度；有状态服务不进节点池

参考放置方案



节点	CPU Σrequest	内存 Σrequest	内存 Σlimit	剩余（request 口径）
节点 1	5.7	11.5 GB	14 GB	2.3 核、4.5 GB
节点 2	5.2	11 GB	11 GB	2.8 核、5.0 GB
节点 3	5.2	11 GB	11 GB	2.8 核、5.0 GB

规则

Tomcat：按节点的拓扑分布（maxSkew 1，DoNotSchedule），高优先级

批处理：低优先级、Burstable

MySQL：专用节点，不在池内；Nginx 在池外

效果与代价

「容量够」的可用性约 99.97%；坏一个节点仍有 4 个 Tomcat，每个 2.55 核

节点由 2 个增至 3 个，多 50%；平时更浪费，留给 L-11

结论 放置方案是配额表（L-09）加故障域的产物，副本数由放法决定

本讲小结

- **放得下：调度按 request 多维装箱**
  - 最先用完的那一维决定能放几个，其余的剩余是碎片
- **坏得起：按故障域放置，可用性按「容量够」计算**
  - 副本数 = 所需数量 + 单个故障域内最多的数量
- **补得回：流量秒级切走，容量分钟级补回，还要有空位**
  - 优先级决定放不下时谁让位
- **技术路线：三问的答案逐步交给平台**
  - 按负载类型选路线，几条路线可以并存

判断标准 这份资源放在哪儿？坏了一处，丢多少、多久补回、谁让位？

课后思考与参考资料

课后思考

1. 为小组项目画一张放置图：每个节点上放了哪些实例，各维度的 Σrequest 是多少，哪一维先满？

2. 小组项目若部署在 3 台云主机上，坏一台时「容量够」吗？按第 12 页的方法计算可用性。

3. 查阅你们所用的云平台：两台云主机会不会落在同一台物理宿主机上？平台提供什么手段避免？

参考资料

Kubernetes docs：Kubernetes Scheduler；Scheduling Framework；Resource Bin Packing；Assigning Pods to Nodes；Pod Topology Spread Constraints；Taints and Tolerations；Pod Priority and Preemption；Disruptions

A. Verma et al., Large-scale cluster management at Google with Borg, EuroSys 2015

B. Burns et al., Borg, Omega, and Kubernetes, ACM Queue, 2016

A. Ghodsi et al., Dominant Resource Fairness: Fair Allocation of Multiple Resource Types, NSDI 2011

华为云文档：弹性云服务器（云服务器组）、云硬盘、云容器引擎、云容器实例、函数工作流

感谢聆听

饮 水 思 源　爱 国 荣 校